`IHostedService` is available in ASP.NET Core, so you can run background tasks in your Razor Pages and minimal API applications. However, sometimes all you want is the background task; you don't need any UI. For those cases, you can use the generic `IHost` abstraction without having to bother with HTTP handling at all.

## *34.2 Creating headless worker services using IHost*

In this section you'll learn about worker services, which are ASP.NET Core applications that do not handle HTTP traffic. You'll learn how to create a new worker service from a template and compare the generated code with a traditional ASP.NET Core application. You'll also learn how to install the worker service as a Windows Service or as a systemd daemon in Linux.

In section 34.1 we cached exchange rates based on the assumption that they're being consumed directly by the UI part of your application, such as by Razor Pages or minimal API endpoints. However, in the section 34.1.2 example we saved the rates to a database instead of storing them in-process. That raises the possibility that other applications with access to the database will use the rates too. Taking that one step further, could we create an application which is responsible only for caching these rates and has no UI at all?

Since .NET Core 3.0, ASP.NET Core has been built on top of a generic `IHost` implementation, as you learned in chapter 30. The `IHost` implementation provides features such as configuration, logging, and DI. ASP.NET Core adds the middleware pipeline for handling HTTP requests, as